VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA STROJNÍHO INŽENÝRSTVÍ

FACULTY OF MECHANICAL ENGINEERING

ÚSTAV MECHANIKY TĚLES, MECHATRONIKY A BIOMECHANIKY

INSTITUTE OF SOLID MECHANICS, MECHATRONICS AND BIOMECHANICS

MODULÁRNÍ SYSTÉM PRO AUTOMATIZOVANÉ ŘÍZENÍ MUZEJNÍCH EXPOZIC

MODULAR SYSTEM FOR THE AUTOMATED CONTROL OF MUSEUM EXHIBITS

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE Andrej Žabka

AUTHOR

VEDOUCÍ PRÁCE Ing. Michal Bastl, Ph.D.

SUPERVISOR

BRNO 2026

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA STROJNÍHO INŽENÝRSTVÍ

Zadání bakalářské práce

Ústav:	Ústav mechaniky těles, mechatroniky a biomechaniky
Student:	**Andrej Žabka**
Studijní program:	Mechatronika
Studijní obor:	bez specializace
Vedoucí práce:	**Ing. Michal Bastl, Ph.D.**
Akademický rok:	2025/26

Ředitel ústavu Vám v souladu se zákonem č.111/1998 o vysokých školách a se Studijním a zkušebním řádem VUT v Brně určuje následující téma bakalářské práce:

Modulární systém pro automatizované řízení muzejních expozic

Stručná charakteristika problematiky úkolu:

Student se bude zabývat návrhem a realizací modulárního systému pro automatizované řízení muzejních expozic. Úkolem je vytvořit řešení, které umožní spouštění časovaných scén v jedné místnosti a zajistí ovládání různých zařízení, jako je osvětlení, motory, aktuátory či speciální efekty. Systém bude založen na využití platformy Raspberry Pi jako centrálního řídicího prvku a modulů ESP32 jako distribuovaných periferních jednotek. Komunikace bude realizována prostřednictvím protokolu MQTT a strukturovaného formátu JSON, který umožní snadno definovat jednotlivé scény a jejich parametry. Výsledkem práce bude funkční vzorek systému pro jednu místnost, který bude demonstrovat možnosti navrženého řešení, jeho modularitu a připravenost k rozšíření.

Cíle bakalářské práce:

1. Provedení rešerše problematiky, zejména protokolu MQTT, formátu JSON a využití platforem Raspberry Pi a ESP32 v oblasti IoT.
2. Návrh architektury systému se zajištěním modularity a možnosti budoucího rozšíření.
3. Návrh a implementace hardwarové a softwarové části systému pro jednu místnost s možností spouštění scén.
4. Experimentální ověření funkčnosti systému na demonstrační úloze.

Seznam doporučené literatury:

[1] VALÁŠEK, Michael. Mechatronika. Praha: České vysoké učení technické, 1995. ISBN 80-01-01276-X.

[2] HEROUT, Pavel. Učebnice jazyka C. 1. vydání. Brno: Computer Press, 2004. ISBN 80-251-0452-3

[3] MALÝ, Martin. ESP32 prakticky : od základních obvodů k pokročilým aplikacím. 1. vydání. Praha : CZ.NIC, z.s.p.o., 2024. 525 s. ISBN 978-80-88168-79-9.

Termín odevzdání bakalářské práce je stanoven časovým plánem akademického roku 2025/26

V Brně, dne

L. S.

prof. Ing. Jindřich Petruška, CSc. ředitel ústavu	doc. Ing. Jiří Hlinka, Ph.D. děkan fakulty

Abstrakt

Tato bakalárska práca sa zaoberá návrhom a realizáciou modulárneho riadiaceho systému pre interaktívne múzejné expozície s využitím platforiem Raspberry Pi a ESP32...

Summary

This bachelor thesis deals with the design and implementation of a modular control system for interactive museum exhibits using Raspberry Pi and ESP32 platforms...

Klíčová slova

IoT, Raspberry Pi, ESP32, MQTT, múzeum, riadiaci systém, mechatronika

Keywords

IoT, Raspberry Pi, ESP32, MQTT, museum, control system, mechatronics

Bibliografická citace

Citace tištěné práce:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026.

Citace elektronického zdroje:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Online, bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026. Dostupné z: https://www.vut.cz/studenti/zav-prace/detail/174036. [cit. 2026-03-07].

Týmto čestne vyhlasujem, že som bakalársku prácu vypracoval samostatne...

Andrej Žabka

Brno . . . .

Na tomto mieste by som chcel poďakovať svojmu vedúcemu práce za odborné vedenie...

Andrej Žabka

Obsah

1 Úvod **9**

2 Súčasný stav problematiky **10**
2.1 Vývoj výstavníctva a interaktívne expozície . . . 10
2.2 Architektúry riadiacich systémov . . . 10
2.3 Existujúce prístupy k riadeniu expozícií . . . 11
2.3.1 Priemyselné riadiace systémy (PLC) . . . 11
2.3.2 Profesionálne show-control systémy . . . 11
2.3.3 Otvorené IoT a SBC systémy . . . 12
2.4 Komunikačné protokoly . . . 12
2.4.1 Prenosové médiá: drôtové vs. bezdrôtové . . . 12
2.4.2 Aplikačné protokoly . . . 13
2.5 Hardvérové platformy pre IoT . . . 13
2.5.1 Jednodoskové počítače (SBC) . . . 14
2.5.2 Mikrokontroléry pre koncové uzly . . . 14
2.6 Výkonová elektronika a riadenie fyzických efektov . . . 14
2.6.1 Riadenie jednosmerných (DC) motorov . . . 14
2.6.2 Spínanie záťaží a reléové spínače . . . 15
2.7 Dátové formáty pre konfiguráciu a prenos . . . 15
2.8 Prístupy k riadiacej logike expozície . . . 16
2.9 Zhrnutie rešerše . . . 16

3 Návrh architektúry systému **17**
3.1 Požiadavky na systém . . . 17
3.2 Logická a sieťová koncepcia . . . 17
3.3 Hardvérová voľba a sieťový prenos . . . 18
3.4 Princíp softvérového jadra (Scene Engine) . . . 19
3.5 Architektúra komunikačnej vrstvy . . . 19

4 Implementácia hardvérovej časti **21**
4.1 Centrálna jednotka – Raspberry Pi . . . 21
4.2 Distribuované uzly – ESP32 . . . 21
4.2.1 Uzol ovládania jednosmerných motorov . . . 22
4.2.2 Uzol spínacích relé . . . 22
4.2.3 Senzorický tlačidlový uzol . . . 23

5 Implementácia softvérovej časti **24**

5.1 Firmvér ESP32 . . . 24
5.1.1 Spoločný základ firmvéru . . . 24
5.1.2 Tlačidlový uzol . . . 25
5.1.3 Motorický uzol . . . 25
5.1.4 Reléový uzol . . . 26
5.2 Backend Raspberry Pi . . . 26
5.2.1 Vstupný bod a orchestrácia (MuseumController) . . . 27
5.2.2 Kontajner služieb (ServiceContainer) . . . 27
5.2.3 Stavový automat a scene engine . . . 27
5.2.4 Vrstva médií . . . 28
5.2.5 MQTT vrstva . . . 29
5.2.6 Webový dashboard . . . 29
5.3 Konfigurácia scén – formát JSON . . . 30

6 Experimentálne overenie **31**
6.1 Popis demonštračnej úlohy (Setup) . . . 31
6.2 Testované scenáre a výsledky . . . 31
6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom . . . 31
6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému . . . 31
6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will) . . . 32
6.3 Zhodnotenie overenia . . . 32

7 Záver **33**

Zoznam obrázkov **34**

Zoznam tabuliek **35**

Použitá literatúra **36**

Použitá literatúra **36**

1 Úvod

Moderné [1] výstavníctvo prechádza v posledných rokoch výraznou zmenou. Návštevníci múzeí a galérií už nevyhľadávajú iba pasívne prezeranie exponátov, ale často očakávajú interaktívny prvok alebo príbehovú líniu, do ktorej môžu aktívne vstupovať. Takýto typ expozícií kladie zvýšené nároky na technickú infraštruktúru. Jednoduché prehrávanie videa v slučke a statické osvetlenie už nepostačujú. Riadiaci systém musí byť schopný reagovať na vstupy návštevníka, vetviť scénu podľa zvolených možností a synchronizovať jednotlivé prvky expozície v reálnom čase, vrátane zmeny osvetlenia, pohybu mechanických častí či spúšťania efektových zariadení.

Realizácia takýchto inštalácií však naráža na viacero technologických a finančných obmedzení. Komerčné show-control riešenia ponúkajú vysokú spoľahlivosť a funkcionalitu, avšak ich obstarávacie náklady sú pre menšie expozície často neúnosné. Jednoduché časovače a lokálne ovládače naopak neposkytujú potrebnú úroveň interaktivity a flexibility. Problematická je aj kabeláž – v historických objektoch býva ťažké, prípadne nežiadúce viesť desiatky metrov vodičov k senzorom a aktuátorom.

Cieľom tejto bakalárskej práce je návrh a implementácia riadiaceho systému, ktorý uvedené problémy rieši kombináciou IoT technológií a vlastnej softvérovej architektúry. Pôvodný koncept bol založený na jednoduchom lineárnom spúšťači efektov. Analýza požiadaviek však ukázala potrebu komplexnejšieho riešenia, ktoré umožňuje vetvenie scén a reakciu na externé udalosti. Výsledkom je systém postavený na stavovom automate, ktorý umožňuje definovať priebeh expozície presnejšie a flexibilnejšie.

Navrhovaná architektúra využíva bezdrôtový model klient–server, ktorý minimalizuje potrebu rozsiahlej kabeláže. Centrálnu riadiacu logiku zabezpečuje minipočítač Raspberry Pi. Ten komunikuje s distribuovanými uzlami na platforme ESP32 prostredníctvom protokolu MQTT. Správanie systému nie je pevne zakompilované vo firmvéri, ale je konfigurovateľné pomocou JSON súborov. Táto vlastnosť umožňuje jednoducho meniť scenár expozície bez zásahu do zdrojového kódu. Funkčnosť systému je v závere práce demonštrovaná na prototypovej miestnosti, ktorá integruje riadenie motorov, osvetlenia, dymostroja a multimediálneho obsahu.

2 Súčasný stav problematiky

Cieľom tejto kapitoly je analyzovať súčasný stav technických prostriedkov používaných v modernom výstavníctve, multimediálnych inštaláciách a automatizácii. Kapitola poskytuje kritický prehľad dostupných hardvérových a softvérových platforiem s dôrazom na ich vhodnosť pre realizáciu interaktívnych expozícií. Zároveň sú diskutované kľúčové technické požiadavky na komunikačné protokoly v distribuovaných systémoch, prístupy k riadiacej logike expozície a vhodné dátové formáty pre konfiguráciu a prenos príkazov.

2.1 Vývoj výstavníctva a interaktívne expozície

Interaktívne prvky v múzeách nie sú novinkou. Prvé mechanické systémy, typicky jednoduché panely s otázkami a odklopiteľnými odpoveďami, sa objavovali už v 70. a 80. rokoch 20. storočia [2]. S nástupom digitálnych technológií sa tieto prvky rozvinuli do systémov zahŕňajúcich dotykové obrazovky, virtuálnu realitu (VR) a rozšírenú realitu (AR), čím sa zásadne zmenili požiadavky na riadiacu infraštruktúru celej expozície [3].

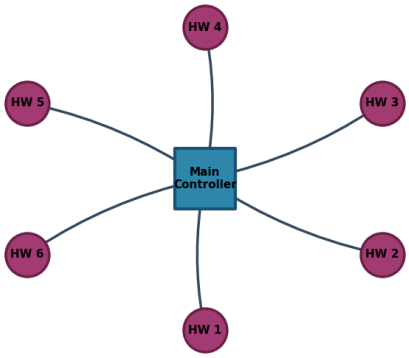
Výskumy zároveň poukazujú na opakujúci sa prevádzkový problém: hoci návštevníci interaktívne prvky oceňujú, ich dlhodobá prevádzkyschopnosť bez stáleho technického personálu je v menších inštitúciách problematická [2]. Systémy postavené na cenovo dostupných IoT platformách tento problém zmierňujú tým, že umožňujú vzdialenú diagnostiku a jednoduchú rekonfiguráciu bez zásahu špecialistu priamo na mieste [2].

2.2 Architektúry riadiacich systémov

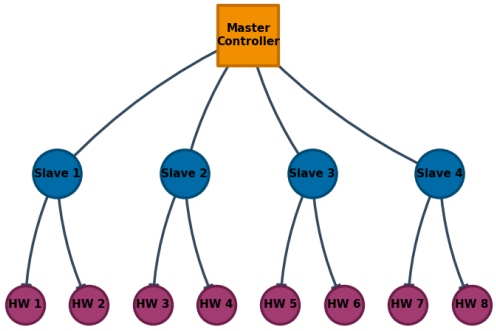
Voľba sieťovej a logickej architektúry riadiaceho systému priamo ovplyvňuje nielen náklady na inštaláciu, ale aj robustnosť a reakčnú dobu celého prostredia [6]. V praxi sa stretávame predovšetkým s dvoma základnými prístupmi:

- **Centralizovaná topológia:** Jedna centrálna jednotka priamo ovláda všetky vstupné a výstupné periférie, zvyčajne cez zapojenie typu hviezda s individuálnymi vedeniami ku každému prvku. Výhodou je jednoduchá správa na jednom mieste, avšak vzniká takzvaný jediný bod zlyhania (*single point of failure*) a v historických budovách je táto topológia problematická kvôli rozsiahlej kabeláži [6].

- **Distribuovaná topológia (Master-Slave):** Inteligencia je rozložená do viacerých uzlov. Centrálna jednotka (master) komunikuje s perifériami (slaves), ktoré riešia svoju vlastnú časť logiky [7]. Každý periférny uzol disponuje vlastným mikrokontrolérom schopným spracovávať lokálne udalosti a komunikovať s centrálnym koordinátorom len pri dôležitých stavových zmenách. Topológia tak umožňuje modulárne rozširovanie systému a redukuje potrebu ťahať kabeláž z jedného riadiaceho bodu [6].



Obrázok 2.1: Centralizovaná topológia



Obrázok 2.2: Distribuovaná topológia

V prostredí distribuovaných systémov sa čoraz viac presadzuje koncept *Cyber-Physical Systems* (CPS), ktoré integrujú výpočtové a fyzické komponenty do jedného celku. Tieto systémy umožňujú presnú synchronizáciu medzi digitálnym obsahom a fyzickými efektmi, čo je nevyhnutné pre vytvorenie imerzívneho zážitku [3]. Výzvou pri návrhu takýchto systémov zostáva predpovedanie priepustnosti a odozvy siete, čo si vyžaduje využitie formalizovaných metód, ako sú konečné stavové automaty (*Finite State Machine*, FSM) [6].

2.3 Existujúce prístupy k riadeniu expozícií

V súčasnosti neexistuje jedno univerzálne riešenie pre implementáciu interaktívnych expozícií. Realizátori si spravidla vyberajú medzi viacerými kategóriami, z ktorých každá má svoje špecifiká, výhody a obmedzenia. Výber konkrétnej platformy závisí od požiadaviek na spoľahlivosť, synchronizáciu multimédií a dostupný rozpočet.

2.3.1 Priemyselné riadiace systémy (PLC)

V oblasti klasickej automatizácie sú dlhoročným štandardom Programovateľné logické automaty (PLC) od výrobcov ako **Siemens, Beckhoff či Mitsubishi** [1]. Ich hlavnou výhodou je determinizmus, teda zaručená odozva systému na vstup v presne definovanom čase [1]. Táto vlastnosť je kritická pre bezpečnostné funkcie, ale pre bežné interaktívne inštalácie v múzeách je často nadbytočná.

PLC sú však primárne navrhnuté na prácu s logickými signálmi a narážajú na výrazné obmedzenia v oblasti multimédií. Absencia natívnych HDMI portov a neschopnosť priamo dekódovať video súbory vyžadujú použitie externých mediálnych prehrávačov [8]. Vysoká cena licencií a hardvéru navyše robí tieto systémy neprístupnými pre menšie kultúrne inštitúcie [2].

2.3.2 Profesionálne show-control systémy

Špecializované systémy ako **Medialon** a **Alcorn McBride** sú navrhnuté priamo pre potreby tematických parkov a múzeí [9, 10]. Na rozdiel od priemyselných PLC ich architektúra nie je postavená na cyklickom vykonávaní kódu, ale na princípe časovej osi (*timeline*), čo umožňuje operátorovi precízne synchronizovať udalosti v čase, podobne ako pri editácii videa [11].

Tieto systémy natívne integrujú protokoly používané v zábavnom priemysle [12]:

- **SMPTE Timecode:** Pre presnú časovú synchronizáciu medzi zvukom, obrazom a fyzickými efektmi.
- **DMX512 / Art-Net:** Pre komplexné riadenie scénického osvetlenia [13].
- **OSC (Open Sound Control):** Pre sieťovú komunikáciu s multimediálnymi servermi.

Napriek vysokej spoľahlivosti je ich hlavnou nevýhodou proprietárna povaha, ktorá vytvára závislosť na dodávateľovi a znemožňuje flexibilné rozširovanie systému o vlastné hardvérové riešenia bez drahých certifikácií [14].

2.3.3 Otvorené IoT a SBC systémy

Treťou, čoraz populárnejšou kategóriou sú otvorené a modulárne systémy postavené na princípoch Internetu vecí (IoT). Tento segment zaznamenal za posledné desaťročie prudký nárast vďaka dostupnosti jednodoskových počítačov (SBC) a výkonných mikrokontrolérov [15]. Na rozdiel od uzavretých systémov táto kategória využíva open-source softvér a štandardné programovacie jazyky (Python, C++), čo poskytuje vývojárom maximálnu slobodu pri tvorbe unikátnych interakcií [5].

Architektúra je typicky rozdelená na dve funkčné úrovne [8]:

- **Výkonné centrálne jednotky (SBC):** Jednodoskové počítače s plnohodnotným operačným systémom (najčastejšie na báze Linuxu) slúžiace na koordináciu celého systému a predovšetkým na náročné multimediálne úlohy.
- **Distribuované periférie:** Koncové prvky (senzory, osvetlenie, mechanika) riadené samostatnými mikrokontrolérmi, ktoré fungujú ako inteligentné sieťové uzly schopné spracúvať dáta lokálne [6].

Využitie platforiem ako Raspberry Pi pre centrálnu logiku a ESP32 pre distribuované uzly umožňuje budovanie systémov, ktoré kombinujú výpočtovú silu potrebnú pre multimédiá s nízkoúrovňovým riadením hardvéru [8]. Tento prístup je nielen finančne efektívny, ale vďaka rozsiahlej komunite a dostupnosti knižníc umožňuje rýchle prototypovanie aj veľmi komplexných nelineárnych scenárov [2].

2.4 Komunikačné protokoly

V distribuovanom systéme je voľba komunikačného protokolu rozhodujúcim faktorom pre plynulosť interakcie [8]. Pri výbere sa zvažujú parametre ako latencia, réžia protokolu a spoľahlivosť doručenia v rámci lokálnej siete [16].

2.4.1 Prenosové médiá: drôtové vs. bezdrôtové

Pri inštaláciách v múzeách je kritickým faktorom spôsob prepojenia komponentov. Tradičné drôtové zbernice (RS-485, Ethernet) ponúkajú vysokú spoľahlivosť a deterministickú komunikáciu. Ich nevýhodou je nutnosť fyzickej kabeláže ku každému prvku, čo môže byť v historických priestoroch problematické [8].

Bezdrôtové technológie (Wi-Fi, Bluetooth, Zigbee) eliminujú potrebu kabeláže a výrazne zvyšujú flexibilitu systému [8]. Mikrokontroléry ESP32 majú vstavané Wi-Fi a priamo

sa pripájajú do lokálnej siete, čo umožňuje jednoducho premiestňovať interaktívne prvky bez nových prípojok [25].

2.4.2 Aplikačné protokoly

Tradičný protokol **HTTP**, hoci je univerzálne podporovaný, sa pre interaktívne IoT systémy javí ako nevhodný [8]. Model request/response vyžaduje od koncového zariadenia neustále dopytovanie (polling) servera, čo vedie k vysokej latencii a zbytočnej záťaži procesora aj sieťového média [17]. Každá HTTP správa nesie objemné textové hlavičky, ktoré sú neefektívne pre prenos krátkych riadiacich príkazov [18].

WebSocket predstavuje lepšiu alternatívu pre aplikácie vyžadujúce obojsmerný tok dát, pretože umožňuje udržiavať trvalé spojenie [8, 19]. Je obzvlášť vhodný pre webové riadiace panely (dashboards), avšak jeho implementácia na jednoduchých mikrokontroléroch je relatívne zložitá a vyžaduje viac pamäťových zdrojov ako optimalizované IoT protokoly [19].

MQTT (*Message Queuing Telemetry Transport*), definovaný ako ISO štandard ISO/IEC 20922:2016, je v súčasnosti najrozšírenejším protokolom pre distribuované IoT systémy [20]. Funguje na modeli *Publish/Subscribe*, kde centrálny sprostredkovateľ (broker) distribuuje správy prihláseným klientom na základe hierarchických tém (topics) [21]. Prenos správ prebieha nad trvalým TCP/IP spojením.

Experimentálne štúdie potvrdzujú, že MQTT v lokálnom sieťovom prostredí výrazne prekonáva alternatívne protokoly z hľadiska latencie a stability [19]. Priemerná odozva 11,04 ms pri použití MQTT QoS 0 je 3,76-násobne nižšia v porovnaní s protokolom WebSocket (41,54 ms), čo je kľúčové pre synchrónne efekty, kde by akékoľvek zreteľné oneskorenie narušilo imerziu návštevníka [19].

Protokol MQTT definuje tri úrovne kvality služieb (QoS), ktoré umožňujú prispôsobiť komunikáciu podľa požiadaviek [21]:

Tabuľka 2.1: Úrovne kvality služby (QoS) v MQTT

QoS	Úroveň	Použitie
0	At most once	Menej kritické dáta (napr. pravidelné senzorické hodnoty), bez potvrdenia
1	At least once	Dôležité správy, možné duplikáty, s potvrdením
2	Exactly once	Kritické povely (napr. pohyb motora), garantované doručenie presne raz

Okrem QoS poskytuje MQTT mechanizmus *Last Will and Testament* (LWT), ktorý umožňuje brokeru automaticky informovať ostatné časti systému, ak dôjde k neočakávanému odpojeniu zariadenia [22]. Tento mechanizmus je v kombinácii so zostatkovými (*retained*) správami kľúčový pre budovanie samoopravných systémov, ktoré dokážu po výpadku napájania okamžite obnoviť svoj stav [22, 21].

2.5 Hardvérové platformy pre IoT

Realizácia riadiaceho systému vyžaduje výber hardvéru pre dve úrovne: centrálnu jednotku určenú na koordináciu systému a multimediálne funkcie, a koncové uzly obsluhujúce senzory, aktuátory a periférie v priestore expozície.

2.5.1 Jednodoskové počítače (SBC)

Jednodoskové počítače (SBC), reprezentované platformou **Raspberry Pi**, sú vhodným riešením pre rolu centrálneho koordinátora expozície. Raspberry Pi ponúka dostatočný výkon pre beh plnohodnotného operačného systému Linux, čo umožňuje využitie pokročilých programovacích jazykov ako Python a správu komplexnej logiky [8].

Pre potreby moderných expozícií je kľúčová podpora multimediálnych rozhraní. Modely Raspberry Pi 4 a 5 disponujú duálnym micro-HDMI výstupom s podporou rozlíšenia 4K a hardvérovou akceleráciou dekódovania videa (H.264, H.265), čo im dáva značnú výhodu oproti priemyselným PLC [23]. Zariadenie zároveň poskytuje 40-pinový GPIO header, cez ktorý možno pripojiť rôzne senzory a aktuátory priamo bez externých adaptérov [23].

2.5.2 Mikrokontroléry pre koncové uzly

Historicky populárna platforma **Arduino Uno** (8-bitový čip ATmega328P) sa v kontexte moderného IoT stáva limitujúcou [24]. S taktom 16 MHz a 2 kB RAM nie je schopná efektívne spracovávať sieťový TCP/IP stack a súčasne parsovať komplexné JSON správy, čo vedie k nestabilite systému pri vyššej sieťovej záťaži [8].

Nástup čipu **ESP32** od spoločnosti Espressif priniesol radikálnu zmenu. Tento 32-bitový systém-na-čipe (SoC) integruje Wi-Fi a Bluetooth konektivitu priamo v module, disponuje dvojjadrovým procesorom s taktom až 240 MHz a 520 kB SRAM [25]. Dvojjadrová architektúra ESP32 je pre interaktívne expozície kritickou výhodou: jedno jadro môže byť vyhradené výhradne pre sieťovú komunikáciu (správa MQTT spojenia, Wi-Fi stack), zatiaľ čo druhé jadro v reálnom čase obsluhuje kritické hardvérové úlohy, ako je PWM riadenie motorov alebo čítanie senzorov. Táto separácia eliminuje nežiaduci jitter v mechanických pohyboch, ktorý by inak vznikal pri spracovávaní sieťovej réžie [26].

Počet priamo ovládaných výstupov môže byť v rozsiahlych inštaláciách nedostačujúci. Na rozšírenie výstupnej kapacity bez obsadenia všetkých GPIO pinov sa využívajú sériové I2C expandéry (napr. PCF8574), ktoré cez dvojvodičovú zbernicu sprístupňujú až 8 dodatočných výstupov z jednej adresy [25].

2.6 Výkonová elektronika a riadenie fyzických efektov

Riadiaci systém musí byť schopný bezpečne a efektívne ovládať výkonové prvky rôznych napäťových hladín — od jednosmerných motorov a nízkonapäťových záťaží (typicky 12 V DC) až po zariadenia pracujúce na sieťovom napätí 230 V AC.

2.6.1 Riadenie jednosmerných (DC) motorov

Na zmenu smeru otáčania a reguláciu rýchlosti DC motorov sa využívajú moduly H-mostíkov. Staršie integrované obvody ako L293D alebo L298N, založené na Darlingtonových tranzistoroch, vykazujú v moderných aplikáciách značné nedostatky [27]. Ich vysoký saturačný úbytok napätia (typicky cez 1,5 V) spôsobuje, že sa pri väčších prúdoch výrazne zahrievajú, čo vyžaduje masívne chladenie a znižuje celkovú účinnosť systému [27].

Moderné H-mostíky prešli na MOSFET tranzistory s extrémne nízkym odporom v zopnutom stave, čím sa minimalizujú tepelné straty a umožňuje sa riadenie prúdov desiatok ampérov takmer bez zahrievania. Pre menšie prúdy sú vhodné kompaktné integrované riešenia ako DRV88xx alebo TB6612FNG, ktoré disponujú ochranou proti prehriatiu,

prepätiu a skratu [28]. Pre väčšie inštalačné motory s vyššími prúdovými nárokmi sa využívajú vysokoprúdové moduly (napr. BTS7960) s maximálnym trvalým prúdom rádovo desiatky ampérov, ktoré sú schopné zvládať kinetické rázy pri rozbehu [28].

2.6.2 Spínanie záťaží a reléové spínače

Pre ovládanie svetelných efektov, dymostrojov alebo iných záťaží v expozíciách je nevyhnutné oddeliť nízkonapäťovú riadiacu logiku od výkonových obvodov. Reálne inštalácie pritom kombinujú záťaže na rôznych napäťových hladinách – bežne 230 V AC (sieťové spotrebiče) aj 12 V DC (LED pásy, malé aktuátory). Pri výbere spínacej technológie sa v praxi porovnávajú dve základné kategórie: **polovodičové relé** (SSR – *Solid State Relay*) a **elektromechanické relé** [29].

SSR nevyužívajú pohyblivé kontakty, čím eliminujú mechanické opotrebovanie, kontaktný oblúk a akustický hluk [29]. Z hľadiska elektromagnetickej kompatibility (EMC) je pri SSR kľúčová technika spínania: *zero-cross* SSR spína záťaž výhradne v momente prechodu sínusoidy nulou, čím minimalizuje prúdové špičky (*inrush current*) a elektromagnetické rušenie, zatiaľ čo *random turn-on* SSR spína okamžite a je vhodnejší pre induktívne záťaže a fázové riadenie výkonu [30, 31]. Hlavnou nevýhodou SSR je obmedzená kompatibilita: väčšina bežne dostupných typov podporuje výhradne striedavé napätie (AC) a vykazuje vyššiu citlivosť na prepätia [29].

Elektromechanické relé spínajú fyzickým rozopnutím a zopnutím kontaktov, vďaka čomu poskytujú úplnú galvanickú izoláciu a sú schopné spínať záťaže rôzneho charakteru – AC aj DC – bez obmedzenia [29]. Nevýhodou sú pohyblivé časti s obmedzenou životnosťou pri vysokej frekvencii spínania a kontaktný oblúk pri induktívnych záťažiach. Vo forme kompletných priemyselných modulov s optickým oddelením vstupov (optočleny) predstavujú elektromechanické relé overené riešenie pre aplikácie s rôznorodými záťažami, nízkou frekvenciou spínania a požiadavkou na prevádzkovú spoľahlivosť [29].

2.7 Dátové formáty pre konfiguráciu a prenos

Pre štruktúrovanie prenášaných dát a ukladanie konfigurácie je nutné zvoliť vhodný formát. V kontexte IoT a webových technológií sa najčastejšie porovnávajú textové formáty XML, YAML a JSON [32].

XML (*eXtensible Markup Language*) je dlhoročným priemyselným štandardom pre výmenu dát. V prostredí embedded systémov je však považované za zastarané kvôli extrémnej verbosite a zložitosti parserov, ktoré neúmerne zaťažujú pamäť mikrokontroléra [32].

YAML (*YAML Ain't Markup Language*) si získal obľubu v oblasti DevOps vďaka podpore komentárov a maximálnej čitateľnosti pre človeka [33]. Avšak YAML parsery pre C++ sú pamäťovo náročné a často chybové pri spracovávaní neštandardných odsadení, čo z neho robí rizikovú voľbu pre real-time komunikáciu na mikrokontroléroch [34].

JSON (*JavaScript Object Notation*) sa stal de facto štandardom pre IoT vďaka optimálnej rovnováhe medzi čitateľnosťou a strojospracovateľnosťou [32]. JSON parsery, ako napríklad vysoko optimalizovaná knižnica ArduinoJson, umožňujú efektívnu prácu s dátami aj na zariadeniach s obmedzenou RAM [35]. Knižnica ArduinoJson vo verzii 7 prináša techniky ako filtrovanie vstupov a zero-copy parsovanie, čím minimalizuje fragmentáciu heap pamäte pri parsovaní JSON správ [36].

Okrem prenosu dát v IoT komunikácii nachádza JSON uplatnenie aj ako **konfiguračný jazyk pre definíciu riadiacej logiky**. Vďaka svojej stromovej štruktúre umožňuje hierarchicky popisovať stavy, časové postupnosti akcií a podmienky prechodov medzi nimi – a to v podobe textového súboru editovateľného bez rekompilácie softvéru [32]. Tento prístup oddeľuje obsah (scenár) od enginu (kód), čo je v kontexte múzejných expozícií prakticky výhodné, keďže scenár môže upravovať aj netechnický personál [33].

2.8 Prístupy k riadiacej logike expozície

Softvérová vrstva riadiaceho systému určuje, akým spôsobom sa definuje a vykonáva scenár expozície. V praxi sa stretávame predovšetkým s dvoma prístupmi, ktoré sa líšia schopnosťou reagovať na externé udalosti.

Najjednoduchším riešením je **lineárny (sekvenčný) prístup**: scenár je pevná séria akcií vykonávaných v pevnom poradí a čase, bez možnosti vetvenia. Takéto systémy sú jednoduché na implementáciu a postačujú pre pasívne inštalácie, kde sa obsah prehráva v slučke bez závislosti od vstupu návštevníka [5]. Ich zásadným obmedzením je, že vlákno čakajúce na uplynutie časovača nie je schopné súčasne reagovať na externé podnety, čo pri zložitejšom scenári vedie k blokovaniu celého systému [6].

Pre interaktívne expozície, kde scenár musí reagovať na vstupy návštevníka (napr. stlačenie tlačidla, dohratý mediálny klip), je vhodnejší prístup založený na **konečnom stavovom automate** (*Finite State Machine*, FSM). V tomto modeli je správanie systému rozdelené do diskrétnych stavov; prechody medzi nimi sú podmienené konkrétnymi udalosťami (*events*) – uplynutím časovača, prijatím MQTT správy alebo dokončením prehrávania média [6]. FSM umožňuje neblokujúce čakanie na viacero rôznych podmienok súčasne, čo je nevyhnutné pre dynamické vetvenie scenára [5].

2.9 Zhrnutie rešerše

Analýza dostupných technických riešení preukázala, že pre špecifické potreby modulárnej, flexibilnej a cenovo dostupnej muzeálnej inštalácie nie sú optimálne ani robustné priemyselné PLC (kvôli vysokej cene a nutnosti centralizovanej kabeláže), ani uzavreté komerčné systémy show-control. Komplexná analýza dostupných technológií a akademických zdrojov potvrdzuje, že pre potreby moderných interaktívnych expozícií je najvhodnejším prístupom vybudovanie distribuovaného IoT systému postaveného na otvorených hardvérových a softvérových platformách [5]. Tento model poskytuje flexibilitu potrebnú pre implementáciu komplexných, nelineárnych naratívov pri zachovaní nízkych nákladov a vysokej udržateľnosti [2].

Z hľadiska komunikačnej vrstvy sa protokol MQTT ukázal ako jednoznačne vhodný pre distribuované systémy s požiadavkou na nízku latenciu a spoľahlivé doručovanie príkazov [19]. Centrálna riadiaca jednotka (SBC) pritom nesie zodpovednosť nielen za MQTT orchestráciu, ale aj za synchrónne prehrávanie audiovizuálneho obsahu v reálnom čase [8, 23]. Pre riadiacu logiku je pri interaktívnych expozíciách vhodný model konečného stavového automatu (FSM), ktorý umožňuje neblokujúce vetvenie scenára na základe externých udalostí [6]. Konfigurácia takéhoto scenára vo formáte JSON poskytuje ľahko editovateľné a strojovo spracovateľné definície stavov a prechodov bez nutnosti zásahu do zdrojového kódu [32].

3 Návrh architektúry systému

Táto kapitola popisuje celkovú architektúru navrhovaného riadiaceho systému. Vychádza z požiadaviek definovaných v predchádzajúcej kapitole a z analýzy dostupných technológií. Popisuje hardvérovú topológiu, softvérovú štruktúru, komunikačnú vrstvu a formát konfiguračných súborov.

3.1 Požiadavky na systém

Táto kapitola definuje hranice projektu a podmienky, ktoré musí výsledný riadiaci systém spĺňať pre zabezpečenie spoľahlivej prevádzky v prostredí expozície.

Z hľadiska **funkčných požiadaviek** sa očakáva najmä bezproblémová integrácia a synchronizácia rôznorodých hardvérových prvkov:

- **Riadenie motora a výkonu:** Primárnou úlohou je riadenie výkonových prvkov, špecificky jednosmerných motorov s riadením ich rýchlosti a smeru otáčania. Ďalšou kľúčovou funkciou je priame spínanie záťaží pracujúcich na sieťovom napätí **230 V** (osvetlenie, dymostroje) a taktiež **12 V**.
- **Multimediálna synchronizácia:** Keďže moderné expozície silne stavajú na audiovizuálnych vnemoch, systém musí zabezpečiť synchronizované prehrávanie zvuku a videa priamo z vlastného úložiska.
- **Interaktivita a časová os (*timeline*):** Systém musí vedieť na základe hardvérových vstupov zmeniť vetvenie prebiehajúcej scény, no zároveň podporovať chronologické časovanie sekvencií.

Medzi **nefunkčné požiadavky** s ohľadom na udržateľnosť patria:

- **Modularita a softvérové oddelenie:** Pre zachovanie oddelenia softvérového inžinierstva od obsahovej tvorby sa úprava logiky (scenára) musí dať vykonať výhradne modifikáciou **textových konfiguračných súborov**. Tým sa odstraňuje nutnosť zásahu do zdrojových kódov Pythonu.
- **Dostupnosť (Cost-effectiveness):** Vzhľadom na finančné limity inštalácií musí byť systém postavený na komerčne dostupných, cenovo výhodných mikrokontroléroch a **Internet of Things (IoT)** platformách namiesto drahých priemyselných automatov (PLC).

3.2 Logická a sieťová koncepcia

Na architektúru navrhovaného systému je nevyhnutné nazerať z dvoch odlišných, no vzájomne sa dopĺňajúcich hľadísk: z **logického (výpočtového)** a z **fyzického (sieťového)**.

Z logického hľadiska ide o **distribuovanú architektúru**. Výpočtový výkon a inteligencia nie sú sústredené výhradne v jednom centrálnom procesore, ale sú rozložené do

priestoru priamo k výkonovým prvkom. Koncové periférie postavené na mikrokontroléroch neslúžia len ako pasívne spínače. Naopak, aktívne dekódujú prichádzajúce textové správy (parsovanie reťazcov MQTT payloadu) a priamo generujú špecifické signály (napr. pulznošírkovú moduláciu pre plynulé riadenie otáčok motorov či jednoduché zapnutie/vypnutie spotrebičov pomocou relé.).

Z fyzického a sieťového hľadiska však systém prísne dodržiava **hviezdicovú topológiu (*Star topology*)**. Hoci sú distribuované uzly plne inteligentné, nekomunikujú vzájomne medzi sebou (netvoria *Mesh* sieť). Akýkoľvek dátový tok na sieti vedie striktne obojsmerne len cez centrálny uzol, ktorým je Wi-Fi prístupový bod so sprevádzkovaným MQTT sprostredkovateľom.

[MIESTO PRE OBRÁZOK: Bloková schéma fyzickej topológie hviezdicovej siete s odlíšením centrálneho uzla a okrajových distribuovaných periférií]

3.3 Hardvérová voľba a sieťový prenos

Systém je rozdelený do dvoch samostatných kategórií: na jeden centrálny uzol a na sieť výkonných periférií. Z hľadiska spoľahlivosti a rýchlosti prenosu medzi nimi by bolo ideálne zabezpečiť komunikáciu výhradne plne drôtovým spojením. V praxi však v expozíciách či *escape room* inštaláciách často nie je technologicky alebo esteticky možné naťahovať dodatočné káblové zväzky naprieč miestnosťou. Z tohto dôvodu bolo na distribúciu dátových príkazov z centrály zvolené bezdrôtové Wi-Fi pripojenie.

Dvojúrovňová architektúra bola fyzicky obsadená nasledovným hardvérom:

- **Centrálny uzol („Mozog“):** Hlavnú riadiacu jednotku systému tvorí jednodoskový počítač **Raspberry Pi**. Tento počítač plní úlohu centrálneho uzla, v ktorom je implementovaná hlavná logika riadenia celej miestnosti. Zvolený bol najmä pre svoj dostatočný výpočtový výkon a pamäť, ktoré umožňujú spoľahlivú prevádzku riadiaceho softvéru aj prehrávanie multimediálneho obsahu. Zariadenie zároveň poskytuje natívne rozhrania pre výstup obrazu (HDMI) a zvuku, čo umožňuje priamu obsluhu displeju a reproduktorov. Výhodou je aj dostupnosť sieťových rozhraní (*Wi-Fi* aj *Ethernet*), vďaka ktorým môže systém komunikovať s ostatnými zariadeniami a súčasne hostovať webový ovládací *dashboard* určený pre obsluhu.

- **Distribuované uzly („Svaly a zmysly“):** Na plnenie fyzických úkonov slúži modulárna sieť bezdrôtových mikrokontrolérov **ESP32**, ktoré sú implementované a napájané priamo pri interaktívnych objektoch. Tieto uzly sa na základe svojho zamerania softvérovo rozdeľujú na:
  - **Senzorický uzol** pre hardvérovú detekciu vstupov od používateľov.
  - **Motorický uzol** pre obsluhu fyzického pohybu exponátov (jednosmerné motory).
  - **Reléový uzol** pre spínanie efektov a osvetlenia či iných zariadení pracujúcich na 12 V a sieťovom napätí 230 V.

3.4 Princíp softvérového jadra (Scene Engine)

Ústrednou požiadavkou na softvérové jadro bola schopnosť **vetviť priebeh expozície** na základe reálnych podnetov od návštevníka. Scenár preto nesmel byť pevnou lineárnou sekvenciou akcií - systém musel v ľubovoľnom momente vedieť čakať na vonkajší vstup (napr. stlačenie tlačidla) a podľa jeho výsledku dynamicky zvoliť ďalší smer deja. Klasický prístup blokujúceho vykonávania kódu je na takúto úlohu nevhodný, keďže vlákno čakajúce na jednu udalosť nie je schopné súčasne reagovať na iné podnety z prostredia.

Z tohto dôvodu bolo softvérové jadro navrhnuté ako neblokujúci **Konečný stavový automat (FSM - *Finite State Machine*)**. Priebeh expozície je rozdelený do samostatných **stavov**, pričom systém nepretržite sleduje podmienky prechodu - vypršanie časovača, dokončenie prehrávania média, alebo príchod správy od hardvérovej periférie - a podľa nich plynulo presúva chod scenára do ďalšieho stavu. Podrobná softvérová implementácia tejto architektúry je rozpracovaná v Kapitole 5.

[MIESTO PRE OBRÁZOK: Diagram konečného stavového automatu (FSM) – prechody z onEnter do timeline a čakanie na transition eventy]

3.5 Architektúra komunikačnej vrstvy

Pre výmenu dát medzi centrálnym automatom (v jazyku Python) a distribuovanými periférami (ESP32) bol zvolený **protokol MQTT** (*Message Queuing Telemetry Transport*). Prenos správ prebieha na koncepte *Publish/Subscribe* cez trvalé TCP/IP spojenie s centrálnym sieťovým sprostredkovateľom (*brokerom*).

Navrhnutý model siete implementuje niekoľko kľúčových komunikačných štandardov:

- **Hierarchické adresovanie zariadení:** Každé zariadenie je adresované vlastným topikom v hierarchickom formáte (napr. `room1/motor1`). Raspberry Pi zverejní príkaz na daný topik; ESP32 mikrokontrolér, ktorý má tento topik odoberaný, správu prijme a vykoná požadovanú akciu.

- **Spätná väzba (*feedback*):** Po vykonaní príkazu periférny uzol odošle potvrdzovaciu správu späť na topik s príponou `/feedback` (napr. `room1/motor1/feedback`). Centrála tak vie, že príkaz bol skutočne doručený a fyzicky vykonaný.

- **Detekcia dostupnosti (status + LWT):** Každý ESP32 uzol pri pripojení aktívne zverejní stav `online` na topiku `devices/<id>/status` ako zostatkovú (*retained*) správu - centrála tak vždy pozná aktuálny stav siete. Súčasne je nastavená správa *Last Will and Testament* (LWT), ktorú broker automaticky zverejní ako `offline` na rovnakom topiku v prípade neočakávaného výpadku spojenia.

- **Konfigurácia scény ako externý JSON súbor:** Celá logika scenára - čo má každý exponát vykonať, v akom poradí a za akých podmienok - je definovaná výhradne v externom súbore `.json`, nie priamo v kóde enginu. Obsluha môže scenár

upravovať alebo vymeniť za behu programu bez akéhokoľvek zásahu do zdrojového kódu.

Vďaka tomuto oddeleniu scenára od enginu sa konfigurácia výstavy stáva samostatným textovým dokumentom zrozumiteľným aj pre netechnický personál.

[MIESTO PRE OBRÁZOK: Schéma toku MQTT správ (Publish/Subscribe) medzi Raspberry Pi a koncovými ESP32 uzlami so spätnou väzbou]

4 Implementácia hardvérovej časti

Táto kapitola sa zaoberá implementáciou hardvérovej časti riadiaceho systému. (DOPLNIT)

4.1 Centrálna jednotka – Raspberry Pi

Úlohu hlavného komunikačného a multimediálneho uzla plní minipočítač Raspberry Pi. Z hľadiska fyzickej inštalácie bol umiestnený v tesnej blízkosti audiovizuálnej techniky (priamo za obrazovkou monitora). Toto riešenie minimalizuje dĺžku káblových zväzkov, keďže Raspberry Pi priamo zabezpečuje natívne prehrávanie videa cez rozhranie HDMI ale aj audio výstup cez 3,5 mm jack. Pre bezproblémovú plnú integráciu do distribuovaného systému MQTT postačuje systému len bežný Ethernet alebo Wi-Fi prístup do LAN siete.

Na napájanie tohto centrálneho mozgu je využívaný USB-C adaptér - 5,0 V / 3,0 A.

[MIESTO PRE OBRÁZOK: Fotografia zapojeného Raspberry Pi v rozvádzači s HDMI a LAN káblovou konektivitou]

4.2 Distribuované uzly – ESP32

Riadiaca logika expozície je decentralizovaná do niekoľkých samostatných uzlov postavených na architektúre mikrokontrolérov rodiny ESP32. Tento prístup fyzicky rozdeľuje zodpovednosť do blokov a znižuje potrebu ťahať káblové zväzky naprieč miestnosťou.

Z hľadiska fyzického rozmiestnenia sú riadiace a silové uzly (pre ovládanie motorov a spínanie relé) umiestnené spoločne vo **fyzickom kotle**, ktorý neslúži len ako bežný inštalačný rozvádzač, ale tvorí priamo dominantný estetický exponát expozície. Tento dekoratívny valcový prvok, siahajúci do výšky 60 cm, kombinuje čierny matný povrch s autentickým nitovaným lemom a kovovými nožičkami. Pre umocnenie industriálneho dojmu je v jeho prednej časti integrovaná mriežka prepúšťajúca simulovaný svetelný ráz „ohňa“ a v spodnej časti ukrýva vývod dymostroja na PVC potrubie. Oproti tomu senzorický „tlačidlový“ uzol pre používateľský vstup je namontovaný oddelene pri vstupe do miestnosti.

4.2.1 Uzol ovládania jednosmerných motorov

Tento riadiaci obvod je vyhradený pre ovládanie jednosmerných (DC) motorov, ktoré animujú dominantné dizajnové prvky celej inštalácie (nazvanej **CHRONOS**). Hlavným poháňaným exponátom sú:

- **Hlavné hodiny:** Dekoratívne hodiny s priemerom približne 1,3 m osadené rímskymi číslicami a kovanými ručičkami. Hodiny neplnia reálnu časomernú funkciu; ich vizuálny pohyb dopĺňajú statické dekoratívne ozubené kolesá rozprestierajúce sa do výšky 1,8 m, ktoré dotvárajú industriálny ráz diela.

Hardvérová implementácia uzla pozostáva z nasledujúcich komponentov:

- **Mikrokontrolér ESP32 DevKit V1:** Funguje ako mozog uzla, ktorý spracováva príkazy z MQTT brokera a tým generuje PWM signál pre ovládanie motorov.
- **Výkonový budič BTS7960:** Polovodičový H-mostík slúži na riadenie výkonu motorov. Tento masívne dimenzovaný obvod bezpečne odoláva vysokým prúdovým špičkám pri rozbehoch (až do 43 A).

[MIESTO PRE OBRÁZOK: Schéma zapojenia uzla pre DC Motory (ESP32, BTS7960, 12V zdroj)]

4.2.2 Uzol spínacích relé

Na jednoduché zapínanie a vypínanie ostatných prvkov expozície, ako sú svetlá, budíky či dymostroj, slúži vyhradený reléový uzol:

- **Priemyselný I/O modul Waveshare:** Základom tohto uzla je komerčná hotová doska s čipom **ESP32-S3**. Obsahuje integrovanú podporu pre *Power over Ethernet* (PoE) a zbernicu RS485.
- **10 A mechanické relé:** Modul disponuje ôsmimi nezávislými relé, ktoré obsluhujú rozmanité typy napätia. Spínajú priamo striedavých 230 V na chod dekoratívnej **Edisonovej žiarovky**, **výrobníka dymu** (ktorý vháňa paru do rozvedeného potrubia) a tiež prúd pre **svetlo simulujúce oheň** v kotle. Ďalšou vyhradenou sekciou je spínanie jednosmernej 12 V vetvy určenej na napájanie podsvietenia dvoch oddelených skupín **industriálnych analógových budíkov** a skrytých dizajnových LED pásov.
- **Dôvod nasadenia modulu:** Namiesto lokálne osadených a letovaných polovodičových relé (SSR) bola na zaistenie požiarnej a elektrickej bezpečnosti použitá táto certifikovaná hotová doska. V prípade hardvérového zlyhania v múzejnej prevádzke umožňuje servisným technikom okamžitú, jednoduchú a bezpečnú výmenu „kus za kus“.

[MIESTO PRE OBRÁZOK: Schéma zapojenia relé uzla (Waveshare ESP32-S3, 230V obvody a 12V obvody)]

4.2.3 Senzorický tlačidlový uzol

Fyzický vstup zameraný na interakciu s návštevníkmi je spracovaný samostatným uzlom:

- **Samostatný mikrokontrolér:** Na vstupnom mieste pred stenou je fyzicky osadená inštalačná krabička. Aby si zachovala kompaktné vonkajšie zapojenie, disponuje vlastným DC napájacím konektorom pre privedenie 5 V napätia k vloženej doske **ESP32 DevKit V1**.
- **Antivandal tlačidlo:** Kvôli zabezpečeniu odolnosti pred opakovaným a tvrdým návštevníckym stláčaním bol zvolený priemyselný **19 mm vodotesný kovový spínač**, ktorý je prepojený s krabičkou riadiaceho modulu pomocou leteckého konektora typu **GX12 (4-pin)**. Tento spínač slúži ako prvotný štartovací impulz pre chod interaktívnej scény.
- **Diagnostika zapojenia:** Masívne tlačidlo je vybavené vizuálnym diódovým LED podsvietením v tvare zeleného prstenca. Toto svetlo vizuálne potvrdzuje aktuálny status (napr. úspešné prepojenie s lokálnou Wi-Fi sieťou a MQTT brokerom), čo slúži na rýchlu diagnostiku..

[MIESTO PRE OBRÁZOK: Schéma zapojenia vstupného tlačidla (ESP32, Pull-up rezistory, LED prstenec)]

5 Implementácia softvérovej časti

Táto kapitola opisuje programové vybavenie systému s dôrazom na systémový prístup a minimalizáciu čistých výpisov kódu v prospech názorných blokových schém. Softvér bol rozdelený do dvoch samostatných vrstiev: firmvér mikrokontrolérov ESP32 a backend bežiaci na Raspberry Pi.

5.1 Firmvér ESP32

Všetky tri firmvérové varianty boli implementované v jazyku C++ s využitím frameworku Arduino. Napriek odlišnej hardvérovej zodpovednosti každého uzla zdieľajú spoločnú modulárnu základňu.

5.1.1 Spoločný základ firmvéru

Každý uzol bol pri štarte povinný vykonať rovnakú inicializačnú postupnosť. Najprv bol inicializovaný **hardvérový watchdog timer (WDT)**, ktorý pri úplnom zamrznutí kódu vyvolá núdzový reštart mikrokontroléra. Pre motorický uzol bola časová hranica nastavená na 60 sekúnd, pre tlačidlový a reléový na 30 sekúnd. Následne prebehlo pripojenie na Wi-Fi sieť a inicializácia OTA (*Over-The-Air*) modulu, ktorý umožňuje bezdrôtové nahrávanie nového firmvéru bez fyzického prístupu k zariadeniu.

Po úspešnom nadviazaní Wi-Fi spojenia bol každý uzol prihlásený na MQTT broker. Súčasťou pripojenia bolo nastavenie správy *Last Will and Testament* s payloadom `offline` a aktívne zverejnenie zostatkovej (*retained*) správy `online` na diagnostický topik `devices/<CLIENT_ID>/status`. Broker tak v prípade výpadku automaticky informuje centrálny systém o strate spojenia.

Konfigurácia siete, MQTT servera a prefixu topikov bola vyčlenená do dedikovaného súboru `config.cpp`. Vďaka tomuto oddeleniu môže byť rovnaký firmvér nasadený na viacero fyzických kusov — zmena miestnosti si vyžaduje úpravu výhradne v tomto súbore.

[MIESTO PRE OBRÁZOK: Bloková schéma spoločného inicializačného toku (WDT → HW → WiFi → OTA → MQTT → Loop) zdieľaného všetkými troma ESP32 variantmi]

5.1.2 Tlačidlový uzol

Senzorický uzol bol implementovaný ako čisto vysielací (*publish-only*) modul — neprijímal žiadne príkazy a jeho jedinou úlohou bolo detekovať stlačenie fyzického tlačidla a odoslať trigger scény.

- **Čítanie vstupu:** Tlačidlo bolo zapojené na `GPIO 32` v režime `INPUT` s externým pull-up rezistorom. Logická úroveň `LOW` na pine signalizuje stlačenie (spojenie s GND).
- **Debounce filter:** Na odstránenie mechanického šumu pri stlačení bol implementovaný softvérový debounce filter s časovou konštantou 60 ms. Stavová zmena bola akceptovaná len po uplynutí tejto doby od poslednej detekovanej zmeny.
- **Cooldown ochrana:** Po každom platnom odoslaní triggra bol aktivovaný blokačný interval 4 000 ms. Počas tejto doby boli všetky ďalšie stlačenia ignorované, čím sa predchádzalo rapidnemu odosielaniu MQTT príkazov pri dlhšom podržaní tlačidla alebo rýchlom opakovanom stláčaní.
- **Odoslaný príkaz:** Po úspešnom prechode obidvoma filtrami bol na topik `room1/scene` odoslaný payload `START`, čím bol spustený predvolený scenár na centrálnom Raspberry Pi.

5.1.3 Motorický uzol

Motorický uzol sa prihlásil na príjem príkazov na topiky `room1/motor1`, `room1/motor2` a `room1/STOP`.

- **Parsovanie payloadu:** Prijatý textový reťazec bol rozparsovaný na súčasti príkazu. Formát `ON:<rýchlosť>:<smer>[:<čas_rozbehu>]` umožňoval v jednej správe definovať cieľovú rýchlosť (0–255 pre 8-bitové PWM), smer otáčania (`L`/`R`) a voliteľný čas plynulého rozbehu v milisekundách. Podporované boli aj samostatné príkazy `SPEED:<hodnota>`, `DIR:<smer>` a `OFF`.
- **PWM generovanie:** Fyzické riadenie výkonu prebiehalo cez hardvérové PWM na frekvencii 20 kHz s 8-bitovým rozlíšením. Pre motor 1 boli využité piny `18` (ľavý), `19` (pravý) a `21` (enable). Pre motor 2 piny `27`, `26` a `25`.
- **Plynulý nábeh (*smooth ramping*):** Aby sa predchádzalo mechanickým rázom pri náhlom spustení pohonu, nebola cieľová PWM hodnota nastavená okamžite. V každom iteračnom kroku hlavnej slučky (s oneskorením 100 ms) sa aktuálna hodnota rýchlosti menila o krok 2 smerom k cieľovej hodnote. Pri definovanom čase rozbehu bola rýchlosť interpolovaná lineárne po dobu zadanú v príkaze.
- **Zmena smeru za behu:** Ak prišiel príkaz na zmenu smeru počas pohybu, firmware automaticky spustil spomalenie na nulovú rýchlosť, až potom prepol polaritu H-mostíka a naštartoval rozjazd v novom smere. Tým sa zabránilo vzniku skratových prúdov v obvode budiča.
- **Spätná väzba:** Po vykonaní každého príkazu bol na topik s príponou `/feedback` odoslaný payload `OK` alebo `ERROR`.

[MIESTO PRE OBRÁZOK: Vývojový diagram logiky motorického firmvéru – parsovanie payloadu, logika smooth rampingu a detekcie zmeny smeru]

5.1.4 Reléový uzol

Reléový uzol obsluhoval výstupné zariadenia rozdelené do konfiguračného poľa `DEVICES[]`, pričom každý záznam obsahoval názov MQTT topiku, index bitu na I2C expandéri, príznak invertovanej logiky a voliteľný čas automatického vypnutia.

- **I2C expandér:** Keďže počet dostupných GPIO pinov čipu ESP32-S3 nepostačoval pre priame riadenie ôsmich reléových kanálov, boli fyzické výstupy riadené cez I2C expandér (adresa `0x20`, SDA pin `42`, SCL pin `41`). Stav všetkých ôsmich bitov bol do expandéra zapísaný v jednej I2C transakcii pri každej zmene.
- **Auto-off:** Pre kanál `effect/smoke` (index 3) bol nastavený automatický časovač 12 000 ms. Po jeho uplynutí bol kanál vypnutý bez nutnosti príkazu z centrály, čím sa zabránilo prehriatiu dymostroja.
- **Efektové skupiny:** Okrem priameho ovládania jednotlivých zariadení bol implementovaný modul `effects_manager`, ktorý umožňoval skupinové blikanie zariadení s náhodným časovaním. Skupina `group1` (kanály `light/4` a `light/5`) generovala blesky s trvaním 75–500 ms a pauzami 150–1 500 ms. Skupina `alone` (kanál `light/1`) produkovala kratšie záblesky 60–100 ms s dlhšími pauzami 2 000–5 000 ms. Každé zariadenie v skupine blikalo nezávisle, so staggerovaným štartom.
- **Stavová LED:** Modul `status_led` priebežne signalizoval stav systému prostredníctvom adresovateľnej NeoPixel diódy na pine `38`: plynulé „dýchanie“ v zelenej farbe indikuje plnú prevádzku (Wi-Fi + MQTT), oranžové blikanie signalizuje výpadok MQTT spojenia a rýchle červené blikanie indikuje stratu Wi-Fi siete. Počas OTA aktualizácie svietila dióda trvalo tyrkysovo.
- **Ochrana pri nečinnosti:** Ak nebol prijatý žiadny príkaz počas 180 000 ms, uzol automaticky vypol všetky aktívne zariadenia a zastavil efektové skupiny.

[MIESTO PRE OBRÁZOK: Bloková schéma reléového firmvéru – tok spracovania MQTT správy, I2C zápis a logika effects_managera]

5.2 Backend Raspberry Pi

Centrálny riadiaci softvér bol implementovaný v jazyku Python 3 ako viacvláknová aplikácia bežiaca nepretržite ako systémová `systemd` služba.

5.2.1 Vstupný bod a orchestrácia (MuseumController)

Koreňom celej aplikácie bola trieda **MuseumController** v súbore `main.py`. Po štarte vykonala nasledujúce kroky:

- **Bootstrap logging:** Pred načítaním konfigurácie bolo aktivované minimálne protokolovanie do súboru, ktoré zachytávalo prípadné chyby pri načítaní konfigurácie.
- **Načítanie konfigurácie:** Trieda `ConfigManager` načítala hodnoty zo súboru `config/config.ini` a vystavila ich ako štruktúrované slovníky. Ak súbor chýbal, bol automaticky skopírovaný zo šablóny `config.ini.example`.
- **Inicializácia služieb:** Všetky podsystémy boli vytvorené prostredníctvom triedy `ServiceContainer` v definovanom poradí závislostí.
- **Prepojenie callbackov (*wiring*):** Po inicializácii boli registrované spätné volania medzi podsystémami – napríklad MQTT handler bol napojený na callback spustenia scény (`on_button_press`) a `SceneParser` bol prepojený s MQTT message handlerom pre príjem prechodových udalostí.
- **Správa vlákien:** Spustenie scény prebiehalo v samostatnom démonickom vlákne, čím hlavná slučka zostala responzívna pre systémové signály `SIGINT` a `SIGTERM`.

[MIESTO PRE OBRÁZOK: UML diagram tried pre MuseumController, ServiceContainer a SceneParser so znázornením závislostí a callbackov]

5.2.2 Kontajner služieb (ServiceContainer)

Na zabezpečenie modularity a testovateľnosti bol zvolený vzor **kontajnera služieb** (*Service Container*). Namiesto priameho vytvárania inštancií tried vo vnútri `MuseumController` bola zodpovednosť za inicializáciu delegovaná na triedu `ServiceContainer`. Tá vytvárala komponenty v striktnom poradí závislostí: najprv `AudioHandler` a `VideoHandler`, potom MQTT package (`MQTTDeviceRegistry` → `MQTTFeedbackTracker` → `MQTTMessageHandler` → `MQTTClient`), nakoniec `SystemMonitor` a `ButtonHandler`. V prípade zlyhania ľubovoľného komponentu bol nastavený na `None` a systém pokračoval v degradovanom režime bez danej funkcionality.

5.2.3 Stavový automat a scene engine

Jadro riadiacej logiky tvorili štyri spolupracujúce triedy:

- **StateMachine:** Načítala a validovala JSON súbor scény v dvoch krokoch. Prvým bol štrukturálny krok prostredníctvom JSON schémy (knižnica `jsonschema`), druhým logická validácia – overenie, že všetky cieľové stavy prechodov (`goto`) sú skutočne definované v poli `states`. Po úspešnej validácii spravovala `StateMachine`

aktuálny stav, časovač stavu a celkový čas scény. Pri každej zmene stavu bol voliteľne spustený callback `on_state_change`, ktorý informoval webový dashboard o prechode.

- **SceneParser:** Koordinoval celý životný cyklus scény. Pred spustením prehľadal stromovú štruktúru JSON a identifikoval všetky audio súbory. Súbory s prefixom `sfx_` boli predbežne načítané do RAM (*preloading*), čím sa eliminovalo oneskorenie pri ich prehrávaní počas scény. V hlavnej slučke volal metódu `process_scene()` každých 20 ms (`scene_processing_sleep`), čím bola zaistená dostatočná reaktivita na prechodové udalosti.
- **StateExecutor:** Vykonával akcie stavov – položky `onEnter`, `onExit` a `timeline`. Dispatchovanie jednotlivých typov akcií bolo realizované pomocou slovníka handlerov (`action_handlers`), čo umožňuje pridanie nového typu akcie bez zmeny ostatného kódu. Aktuálne registrované typy boli `mqtt`, `audio` a `video`.
- **TransitionManager:** Uchovával udalosti v troch thread-safe frontách (deque) s maximálnou kapacitou 50 položiek: MQTT udalosti, ukončenia audia a ukončenia videa. V každom iteračnom kroku vyhodnocoval zoznam prechodov aktuálneho stavu voči obsahu front. Bolo podporovaných päť typov prechodov: `timeout` (uplynutie časovača), `audioEnd` (dokončenie prehrávania zvuku), `videoEnd` (dokončenie videa), `mqttMessage` (príchod definovanej MQTT správy) a `always` (okamžitý prechod). Spotrebovaná udalosť bola z fronty odstránená, čím sa predchádzalo jej opakovanému vyhodnoteniu.

[MIESTO PRE OBRÁZOK: Vývojový diagram jedného iteračného kroku scene engine – od volania process_scene() cez check_transitions() po execute_onEnter() nového stavu]

5.2.4 Vrstva médií

- **AudioHandler:** Zvukový modul využíval knižnicu `pygame.mixer` s 32 mixérovými kanálmi pre súčasné prehrávanie viacerých zvukových efektov. Implementované boli dve nezávislé playback cesty: súbory s prefixom `sfx_` boli pri štarte scény načítané do RAM ako objekty `pygame.mixer.Sound` a prehrávané bez diskového prístupu. Ostatné súbory boli streamované priebežne z disku cez `pygame.mixer.music`. Na detekciu prirodzeného konca prehrávania (pre prechod `audioEnd`) bola v každom kroku voliteľná metóda `check_if_ended()`, ktorá porovnávala aktuálny stav mixéra s predchádzajúcim tikom.
- **VideoHandler:** Video bolo prehrávané cez externý proces `mpv` spustený v celoobrazovkovom režime s hardvérovým dekódovaním (`hwdec=rpi4-mmal`). Komunikácia s procesom prebiehala výhradne cez UNIX IPC socket (`/tmp/mpv_socket`) formátom JSON príkazov. Zdravie procesu bolo overované každých 60 sekúnd zaslaním testovacieho príkazu; pri neodpovedi alebo páde procesu bol vykonaný automatický reštart (maximálne 3 pokusy). Keď nebolo prehrávané žiadne video, bol zobrazený čierny idle obrázok (`black.png`).

5.2.5 MQTT vrstva

MQTT package bol rozdelený do štyroch tried so striktne oddelenou zodpovednosťou:

- **MQTTClient:** Wrapper nad knižnicou `paho-mqtt` zabezpečujúci pripojenie, opakované pokusy o reconnect a publikovanie správ. Po úspešnom pripojení sa automaticky prihlásil na štyri skupiny topikov: `devices/+/status`, `room1/+/feedback`, `room1/scene` a `room1/#`.

- **MQTTMessageHandler:** Smerovací modul, ktorý každú príchodziu správu vyhodnotil podľa prioritného poradia: správy `devices/.../status` boli odovzdané registru zariadení, správy s príponou `/feedback` trackeru spätnej väzby, payload `START` na topiku `room1/scene` spustil callback scény, payload s názvom JSON súboru na topiku `room1/start_scene` spustil pomenovanú scénu, a všetky ostatné správy boli odovzdané `TransitionManager`-u ako potenciálne prechodové udalosti.

- **MQTTFeedbackTracker:** Pre každý odoslaný príkaz bol spustený jednovláknový časovač s timeoutom 1 s. Ak počas tejto doby neprišla zodpovedajúca správa na topik `<pôvodný_topik>/feedback`, bol zaznamenaný timeout. Sledovanie bolo aktívne výhradne počas behu scény.

- **MQTTDeviceRegistry:** Uchovával posledný známy stav každého ESP32 uzla. Zostatková (*retained*) správa `online` pri pripojení bola zámerene ignorovaná, aby sa predišlo falošne pozitívnemu stavu pri štarte. Zariadenia bez aktualizácie stavu dlhšej ako 25 sekúnd boli automaticky označené ako `offline`.

[MIESTO PRE OBRÁZOK: Diagram toku príchodziej MQTT správy cez MQTTMessageHandler – prioritné smerovanie do štyroch vetiev]

5.2.6 Webový dashboard

Riadiace rozhranie pre obsluhu múzea bolo implementované ako webová aplikácia dostupná na porte 5000. Backend tvorilo microframework **Flask** s rozšírením **Socket.IO** pre asynchrónny obojsmerný prenos udalostí.

- **Autentifikácia:** Všetky HTTP endpointy aj WebSocket spojenia boli chránené *HTTP Basic Authentication*.

- **REST API:** Definovaných bolo niekoľko skupín endpointov: správa scén (`/api/scenes`, `/api/run_scene`, `/api/stop_scene`), manuálne MQTT príkazy (`/api/mqtt/send`),

správa médií (`/api/media/audio`, `/api/media/video`) a systémové operácie (`/api/system/reboo` `/api/system/shutdown`).

- **WebSocket eventy:** Pri každej zmene stavu scény alebo pripojenia zariadenia bol vysielaný (*broadcast*) príslušný Socket.IO event (`status_update`, `device_status_update`, `scene_progress`) všetkým pripojeným klientom v reálnom čase.
- **Logovanie:** Vlastný handler `WebLogHandler` zachytával záznamy zo všetkých logerov systému a prenášal ich cez Socket.IO event `new_log` priamo do prehliadača obsluhy. História posledných 1 000 záznamov bola udržiavaná v pamäti.

5.3 Konfigurácia scén – formát JSON

Celá choreografia expozície bola definovaná v externom súbore `.json`, čím bolo scenárové obsahu oddelené od kódu enginu. Obsluhujúci personál bol schopný upravovať časovanie a poradie efektov bez zásahu do zdrojového kódu Pythonu či firmvéru.

Koreňová úroveň súboru obsahovala povinné polia `sceneId`, `initialState` a `states`, voliteľne aj `globalEvents`. Globálne udalosti boli vyhodnocované v každom iteračnom kroku bez ohľadu na aktuálny stav - v nasadenej scéne (`fire_story_hw_sequence`) boli definované dve: núdzový stop na MQTT správu `room1/emergency = STOP` a globálny timeout 600 sekúnd ako poistka proti zaseknutiu.

Každý stav mohol obsahovať tri funkčné bloky:

- **`onEnter`:** Pole akcií vykonaných okamžite po vstupe do stavu (spustenie zvuku, odoslanie MQTT príkazu, spustenie videa).
- **`timeline`:** Pole časovaných akcií s atribútom `at` určujúcim čas v sekundách od vstupu do stavu. Umožňovalo orchestráciu efektov v rámci jedného stavu bez nutnosti zavádzať ďalší stav len pre časové oddelenie.
- **`transitions`:** Pole podmienok prechodu do nasledujúceho stavu. Každý prechod obsahoval povinný typ a cieľový stav (`goto`), špeciálne vyhradenú hodnotu `END` signalizovala ukončenie scény.

[MIESTO PRE VÝPIS/SCHÉMU: Zjednodušená vizualizácia štruktúry jedného stavu JSON scény so znázornením onEnter, timeline a transitions blokov]

6 Experimentálne overenie

Táto kapitola prezentuje reálne skúšky navrhnutého systému, ktoré overujú splnenie stanovených cieľov zadania v podmienkach reálnej prevádzky.

6.1 Popis demonštračnej úlohy (Setup)

- **Fyzický prototyp:** Systém bol priamo nainštalovaný a oživený na konkrétnom reálnom exponáte v priestranstve expozície.

- **Zapojené uzly:**

  - Raspberry Pi s bežiacim centrálnym procesom `MuseumController` a pripojenou audiovizuálnou technikou.
  - ESP32 Motorový uzol pripojený na lokálny jednosmerný pohon.
  - ESP32 Reléový uzol spínajúci scénické osvetlenie miestnosti.

6.2 Testované scenáre a výsledky

Overenie stability systému prebiehalo v troch izolovaných rovinách: prevádzka s hardvérom, softvérová výdrž jadra a odolnosť voči sieťovým výpadkom.

6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom

- **Cieľ:** Otestovať plynulosť a asynchrónnu presnosť prechodov definovaných v JSON súbore počas riadenia fyzických aktuátorov priamo v múzeu.

- **Priebeh:** Systém musel prehrať multimediálny audiovizuálny obsah súčasne s presne načasovaným pohybom motorov a dynamickým zhasínaním svetiel.

- **Výsledok:** Všetky akcie boli spustené v časoch definovaných v *timeline*. Komunikácia s mikrokontrolérmi prebehla bez zaznamenaného oneskorenia a synchronizácia audio-video stopy so svetelnými efektmi bola vizuálne konzistentná počas celého trvania testu.

6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému

- **Cieľ:** Overiť odolnosť Raspberry Pi a MQTT brokera proti pretečeniu pamäte a zlyhaniu pri dlhodobom plynulom behu (tzv. *Soak test*).

- **Priebeh:** Centrálny program systému bežal na stole formou *dry-run* testovania v kuse niekoľko dní v laboratórnych podmienkach. Priebežne dochádzalo k ojedinelému spúšťaniu scén a monitorovaniu odosielaných MQTT príkazov na zbernicu.

- **Výsledok:** Proces `MuseumController` zostal počas celej doby testu bez reštartu. Príkazy boli generované na MQTT zbernicu priebežne bez výpadku, čo preukázalo dostatočnú stabilitu softvéru pre nepretržitú prevádzku.

6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will)

- **Cieľ:** Overiť detekčný a oznamovací mechanizmus systému pri neštandardnej strate bezdrôtového spojenia uzlov na Wi-Fi sieti.
- **Priebeh:** Koncovým ESP32 uzlom (*Controller_MOTORS* a *Controller_RELAY*) bolo striedavo fyzicky odpájané a následne vracané napájanie za behu celého systému.
- **Výsledok:** Po obnovení napájania sa uzly autonómne pripojili späť na Wi-Fi sieť a MQTT brokera. Dashboard správne zaznamenal zmenu stavu - prepis indikátora z `offline` na `online` prebehol na základe LWT správy od brokera a následného `online` hlásenia od uzla.

6.3 Zhodnotenie overenia

- **Záver experimentov:** Všetky tri testované scenáre boli úspešne overené. Architektúra distribuovanej hviezdicovej siete sa ukázala ako dostatočne stabilná pre podmienky plného fyzického zaťaženia v prostredí expozície.
- Bezdrôtová komunikácia cez protokol MQTT preukázala reaktivitu postačujúcu na spracovanie hardvérových triggerov bez merateľného oneskorenia, čo vylúčilo potrebu prechodu na drôtové prepojenie.

7 Záver

Záverečné zhrnutie práce posudzujúce, či a ako sa podarilo naplniť pôvodné ciele zadania.

- **Ciele a dosiahnutý stav:**
  - Hlavným cieľom práce bolo navrhnúť modulárny riadiaci systém pre múzejné expozície, ktorý umožní synchrónne ovládanie fyzických efektov (motory, 230 V spotrebiče, osvetlenie) a multimediálneho obsahu z jediného centrálneho uzla.
  - Realizácia potvrdila úspešnú fúziu výpočtovo silného Raspberry Pi a distribuovaných ESP32 sieťových uzlov do jednej funkčnej virtuálnej hviezdy.
- **Hlavné prínosy riešenia:**
  - Najväčším prínosom je odstrihnutie tvrdej programátorskej aplikačnej logiky (Python/C++) od samotného chodu výstavy. Vďaka aplikácii inžinierskeho konečného automatu (FSM) a používaniu textového kontraktu v `.json` formáte dokáže muzeálny personál upravovať časy, kroky a audiovizuálny obsah výstavy bez nutnosti vedieť programovať či využívať drahé licencie priemyselných PLC.
- **Výsledky overenia technológie:**
  - Reálne záťažové testy na exponáte deklarovali, že Publish/Subscribe model (MQTT nad lokálnou LAN/Wi-Fi sieťou) netrpí pre výstavu vnímateľnými reaktančnými oneskoreniami.
  - Architektúra bez problémov prechádzala dlhodobými *Soak* testami a vďaka LWT packetom úspešne ošetrila fyzické odpájanie uzlov.
- **Nedostatky a obmedzenia:**
  - Bezpečnostná vrstva Wi-Fi siete v reálnom múzeu musí byť prevádzkovo dedikovaná len pre exponát, inak hrozí rušenie spôsobené návštevníkmi (kolízie frekvenčného pásma).
  - Nutnosť ručného hardcodovania `BASE_TOPIC_PREFIX` v C++ firmvéri pre každý nový fyzický kus ESP32 mikročipu pri úvodnom *flashovaní*.
- **Návrhy na budúci rozvoj:**
  - Prechod do plne lokalizovanej webovej administrácie (React Dashboard), odkiaľ bude možné tvoriť JSON scény vizuálnym presúvaním blokov (tzv. „Node-Based Editor“).
  - Možnosť plnohodnotnej over-the-air (OTA) aktualizácie firmvéru priamo prostredníctvom odoslania binárneho príkazu cez `Mosquitto` hub na konkrétny MQTT `device id` uzol.

Zoznam obrázkov

2.1 Centralizovaná topológia 11
2.2 Distribuovaná topológia 11

Zoznam tabuliek

2.1 Úrovne kvality služby (QoS) v MQTT 13

Použitá literatúra

[1] AL-JUMAILY, M. Automation in industry using Programmable Logic Controllers (PLCs). *International Journal of Computer and Communication Engineering*. 2015, roč. 6, s. 924. Dostupné z: https://www.computersciencejournals.com/ijccn/article/83/6-1-1-924.pdf

[2] SCHOLARSBANK UOREGON. Putting the "Play"Back into Display: Interactive Exhibits in Small Museums [online]. Eugene: University of Oregon, 2026 [cit. 2026-03-15]. Dostupné z: https://scholarsbank.uoregon.edu/server/api/core/bitstreams/810e909e-35d7-4255-8119-dbcd4cc16104/content

[3] Research on the Strategy and Practical Mechanism of Museum Exhibition Space Design Based on Immersive Experience. In: *Proceedings of ETMHS 2025* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://webofproceedings.org/proceedings_series/ESSP/ETMHS%202025/H27.pdf

[4] Designing Interactive Exhibitions Based on Innovative Narrations Guided by Architectural Space and Digital Technologies. In: *NODEM 2006 Proceedings* [online]. 2006 [cit. 2026-03-15]. Dostupné z: http://repo.nodem.org/uploads/Designing%20interactive%20exhibitions%20based%20on%20innovative%20narrations%20guided%20by%20architectural%20space%20and%20digital%20technologies.pdf

[5] Computational Frameworks for Interactive Multimedia Art Installations in Public Spaces using Real Time Digital Control and Visualization. *PeerJ Computer Science* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://peerj.com/articles/cs-3384.pdf

[6] Modeling and Analysis of Distributed Control Systems: Proposal of a Methodology. *Processes* [online]. 2024, roč. 12, č. 1 [cit. 2026-03-15]. Dostupné z: https://www.mdpi.com/2227-9717/12/1/5

[7] Design and Implementation of Intelligent Audio-Video Interaction and Control Platform Based on Distributed Architecture. In: *MATEC Web of Conferences* [online]. 2021 [cit. 2026-03-15]. Dostupné z: https://www.matec-conferences.org/articles/matecconf/pdf/2021/05/matecconf_cscns20_05002.pdf

[8] AL-SHUHAIB, T. Low-Cost ESP32, Raspberry Pi, Node-Red, and MQTT Protocol Based SCADA System. *IEEE International Conference on Intelligent Systems*. 2020. Dostupné z: https://www.researchgate.net/publication/347154219

[9] What is Medialon? Competitors, Complementary Techs & Usage. *Sumble* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://sumble.com/tech/medialon

[10] Alcorn McBride – Reliable AV Technology for Themed Attractions [online]. 2026 [cit. 2026-03-15]. Dostupné z: https://alcorn.com/

[11] Engineering the Invisible Magic: Alcorn McBride at 40. *InPark Magazine* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.inparkmagazine.com/109alcornmcbride/

[12] SIMPSON, John Huntington. *Show Networks and Control Systems*. 2. vyd. New York: Entertainment Technology Press, 2017. ISBN 978-1138090874.

[13] SOMERVILLE, I. *DMX512 Stage Lighting Basics*. Scribd Document [online]. 2013 [cit. 2025-02-20]. Dostupné z: https://www.scribd.com/document/132762444/DMX-Beginners-Guide-Rev2

[14] BPAV TECHNOLOGY. Crestron vs Extron vs AMX: What Suits Your Project? *BPAV Blog* [online]. 2023 [cit. 2025-02-20]. Dostupné z: https://blog.bpav.global/news-insights/crestron-vs-extron-vs-amx-what-suits-your-project

[15] Interactive Museum Exhibits with Microcontrollers: A Use-Case Scenario. *arXiv* [online]. 2015 [cit. 2026-03-15]. Dostupné z: https://arxiv.org/abs/1509.07626

[16] Evaluating HTTP, MQTT over TCP and MQTT over WEBSOCKET for Digital Twin Applications: A Comparative Analysis on Latency. *IJIRSS* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://www.ijirss.com/index.php/ijirss/article/download/4414/623/7222

[17] MQTT vs. HTTP for IoT. *HiveMQ Blog* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.hivemq.com/blog/mqtt-vs-http-protocols-in-iot-iiot/

[18] Comparative Analysis of Power Consumption between MQTT and HTTP Protocols in an IoT Platform Designed for Remote Real-Time Monitoring. *PMC – PubMed Central* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://pmc.ncbi.nlm.nih.gov/articles/PMC10224120/

[19] Comparative Performance Analysis Between the MQTT and WebSocket Protocols. *eJournal: Komunitas Dosen Indonesia* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://jurnal.kdi.or.id/index.php/bt/article/download/3223/1701/20910

[20] ISO/IEC 20922:2016. *Information Technology – Message Queuing Telemetry Transport (MQTT) v3.1.1* [online]. Geneva: IEC, 2016 [cit. 2026-03-15]. Dostupné z: https://webstore.iec.ch/en/iec_catalog/product/preview/?id=L3B1Yi9wZGYvcHJldmlldy9pbmZvX2lzb2llYzIwOTIye2VkMS4wfWVuLnBkZg==

[21] The Use of MQTT in M2M and IoT Systems: A Survey. *ResearchGate* [online]. 2020 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/publication/346414150_The_Use_of_MQTT_in_M2M_and_IoT_Systems_A_Survey

[22] MQTT Protocol for the IoT – Review Paper. *ResearchGate* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper

[23] Raspberry Pi 4 Model B – Datasheet. *Raspberry Pi Ltd* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

[24] ESP32 vs Arduino: Key Differences, Speed & IoT Uses. *NORVI Blog* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://norvi.io/esp32-vs-arduino-which-microcontroller-should-you-choose/

[25] A Comprehensive Analysis of ESP32 Microcontroller for IoT Applications. *IJNRD* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://ijnrd.org/papers/IJNRD2510255.pdf

[26] ESP32 vs Arduino: Which One Should You Choose? *Nantian Electronics* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.ntchip.com/electronics-news/esp32-vs-arduino

[27] Darlington Transistor vs. MOSFET: Selection Guidelines. *Cadence PCB Design & Analysis* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://resources.pcb.cadence.com/blog/2023-pdarlington-transistor-vs-mosfet-selection-guidelines

[28] MOSFET H-Bridge vs Dedicated Motor Driver Solutions. *ALLPCB* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.allpcb.com/allelectrohub/mosfet-h-bridge-vs-dedicated-motor-driver-solutions

[29] Solid State Relay Guide. *Phidgets Support* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.phidgets.com/docs/Solid_State_Relay_Guide

[30] Zero-Cross or Random Relay: What are the Differences? *Celduc Relais* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.celduc-relais.com/en/zero-cross-or-random-relay-what-are-the-differences/

[31] Beware of Zero-Crossover Switching of Transformers. *TE Connectivity* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.te.com/en/products/relays-and-contactors/electromechanical-relays/intersection/zero-crossover-switching-transformers.html

[32] A Literature Review on Device-to-Device Data Exchange Formats for IoT Applications. *ResearchGate* [online]. 2021 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/publication/349022302_A_Literature_Review_on_Device-to-Device_Data_Exchange_Formats_for_IoT_Applications

[33] YAML vs. JSON: Definitions, Pros & Cons. *Built In* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://builtin.com/software-engineering-perspectives/yaml-vs-json

[34] CELERDATA. YAML, JSON and XML: A Practical Guide to Choosing the Right Format. *CelerData Blog* [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://celerdata.com/glossary/yaml-json-and-xml-a-practical-guide-to-choosing-the-right-format

[35] How to Reduce Memory Usage? – ArduinoJson 7 [online]. 2026 [cit. 2026-03-15]. Dostupné z: https://arduinojson.org/v7/how-to/reduce-memory-usage/

[36] ArduinoJson 7 – Release Notes [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://arduinojson.org/news/2024/01/03/arduinojson-7/